

Java代码审计基础-Struts2基础知识

文章目录

- 一、Struts2介绍
 - Struts2是什么?
 - 如何获取Struts2
 - 如何构建一个基于Struts2的Web项目
 - 如何调试Struts2源码
- 二、Struts2的工作原理
 - Struts2的初始化
 - Struts2的配置管理
 - Struts2处理HTTP请求 (FilterDispatcher)
 - HTTP中的参数如何映射到Action中的?
 - ActionContext与Action?
 - Action处理结束后的动作
 - Response页面跳转
- 三、Struts2的核心对象
 - Container容器
 - ActionContext
 - 其它
- 四、Struts2整体流程图

一、Struts2介绍

1.1、Struts2是什么

Apache Struts 2最初被称为WebWork 2，它是一个简洁的、可扩展的框架，可用于创建企业级Java web应用程序。

Web应用程序开发最经典的模式为MVC，即Model、View、Control；数据模型、视图展现、控制器

也有表示层、业务层、持久层的分层模型，Struts2负责的便是表示层

Struts2是一个框架，但本质只是一个Jar文件，是针对JDK的扩展。

1.2、如何获取Struts2

1、通过官网：<https://struts.apache.org/>

2、通过Maven：IDEA配置Maven，修改项目的pom.xml即可

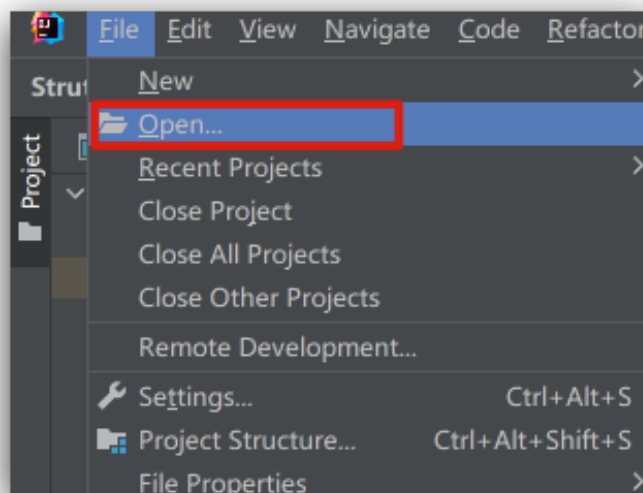
1.3、如何构建一个基于Struts2的Web项目

参考链接：【Java代码审计环境搭建-IDEA安装配置及运行Web环境】

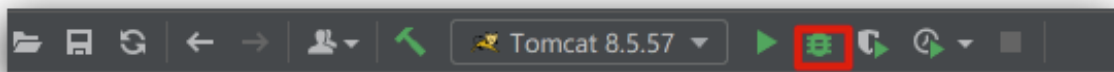
1.4、如何调试Struts2源码

Java的项目类型很多，例如普通的JavaWeb、SpringMVC类型、SSH类型、微服务、Maven类型等等，不同的项目框架类型，导入方式稍有不同，我们在做代码审计时一般是导入项目更多一些，我们以一个简单的Java Web项目导入为例，需要3步骤：

- 1、将项目拷贝到你的IDEA的工作目录
- 2、点击File->Open，选择对应项目即可
- 3、需要重新配置下Tomcat，运行即可

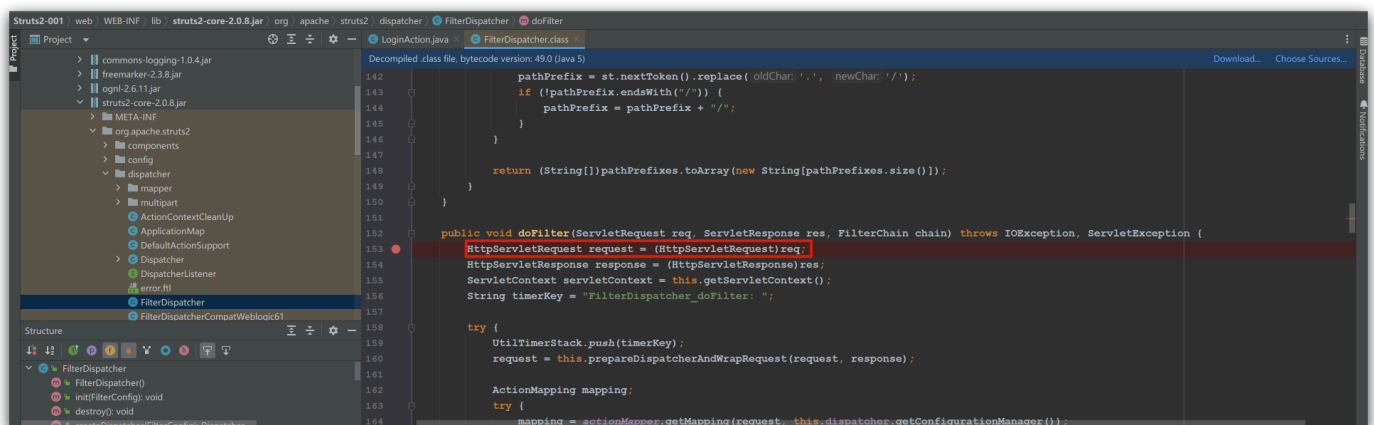


调试代码需要选择IDEA的Debug方式运行项目



然后在需要断点的地方打断点，有两种情况

- 1、自己的写代码，直接打断点即可
- 2、如果是jar包的代码，需要点开jar包，找到对应的类进行断点



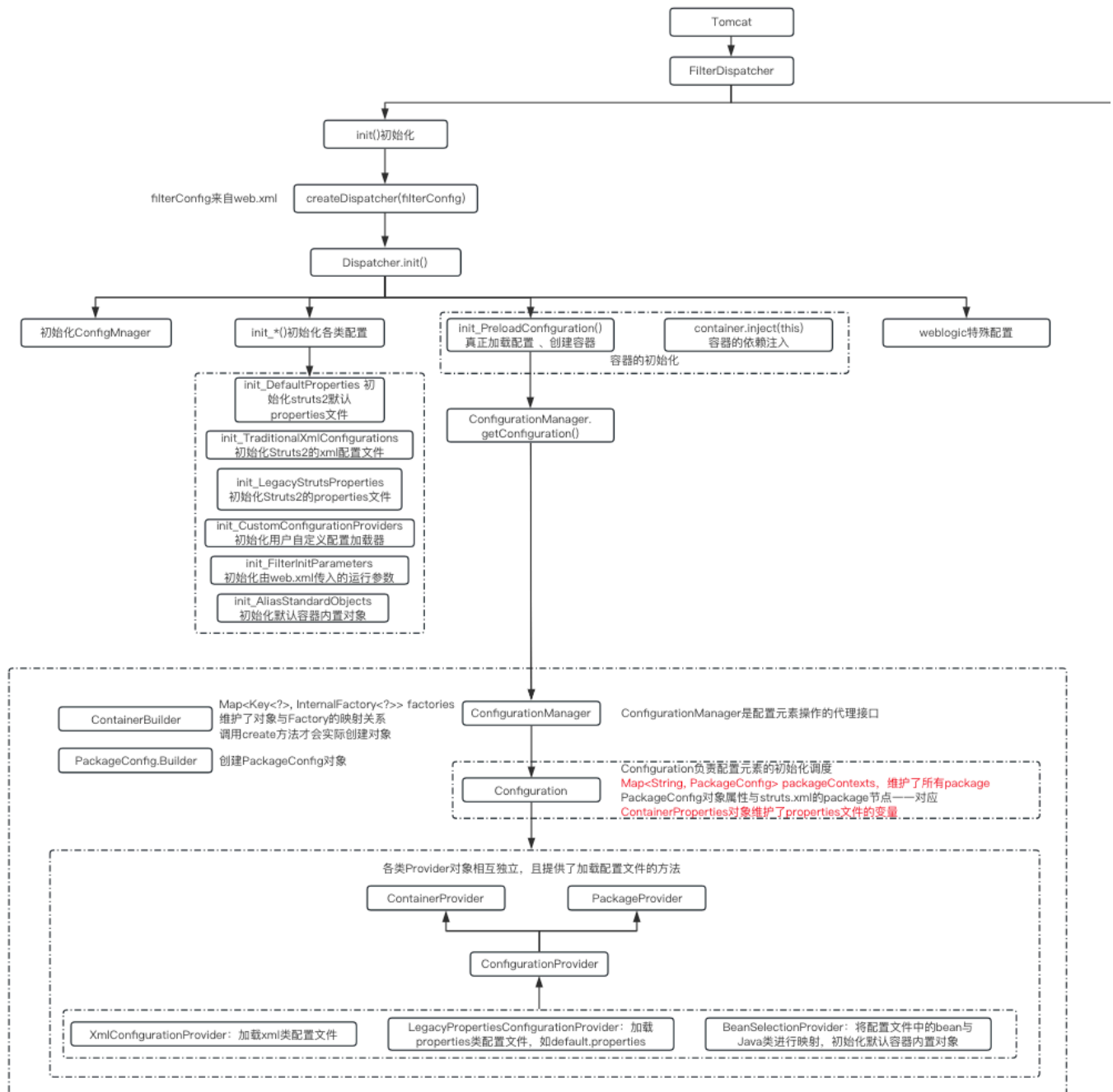
访问对应的HTTP请求，即可在断点处停下进行调试，IDEA常用的快捷键有

- F8 Step Over: 跳过，一行一行的向下走，如果当前行有方法不进入方法。`
- F7 Step Into: 跳入，如果当前有方法，进入方法内部，一般用于进入自定义的方法，不会jdk类库的方法。
- Alt+Shift+F7: Force Step Into :强制跳入，可以进入任何方法，想查看底层源代码可以用这个进入jdk的方法。
- Shift+ F8 : Step Out:跳出，从跳入的方法内退出到方法调用处，这个时候方法已经执行完毕，只是还没有完成赋值
- Alt+ F9: Run to Cursor:运行到光标处，你可以将光标定位到你需要查看的行，然后使用这个功能，代码会运行到光标处，不需要打断点。
- Alt+ F8: Evaluate Expression,计算表达式的值。
- Ctrl+ F5: Return xxx:重新运行程序，会关闭服务后重新启动程序。
- F9: Resume Program:恢复程序，比如，你在第3行和5行各自都打了断点，当前运行到第3行，按F9,则运行到下一个断点即第5行，再按F9，则运行完整个流程，因为后面已经没有断点了。

二、Struts2的工作原理

2.1、Struts2的初始化

首先先看下Struts2的初始化过程图



在FilterDispatcher类中，有个init方法用于初始化

```

52 public void init(FilterConfig filterConfig) throws ServletException {
53     this.filterConfig = filterConfig;
54     this.dispatcher = this.createDispatcher(filterConfig);
55     this.dispatcher.init();
56     String param = filterConfig.getInitParameter("packages");
57     String packages = "org.apache.struts2.static template org.apache.struts2.interceptor.debugging";
58     if (param != null) {
59         packages = param + " " + packages;
60     }
61
62     this.pathPrefixes = this.parse(packages);
63 }
  
```

核心在于this.dispatcher.init()方法，内容如下图，在init方法中对properties、xml文件进行了初始化

```

297 public void init() {
298     if (this.configurationManager == null) {
299         this.configurationManager = new ConfigurationManager( name: "struts");
300     }
301
302     this.init_DefaultProperties();
303     this.init_TraditionalXmlConfigurations();
304     this.init_LegacyStrutsProperties();
305     this.init_ZeroConfiguration();
306     this.init_CustomConfigurationProviders();
307     this.init_MethodConfigurationProvider();
308     this.init_FilterInitParameters();
309     this.init_AliasStandardObjects();
310     Container container = this.init_PreloadConfiguration();
311     this.init_CheckConfigurationReloading(container);
312     this.init_CheckWebLogicWorkaround(container);
313 }

```

例如对struts.xml的初始化代码为this.init_TraditionalXmlConfigurations();代码如下

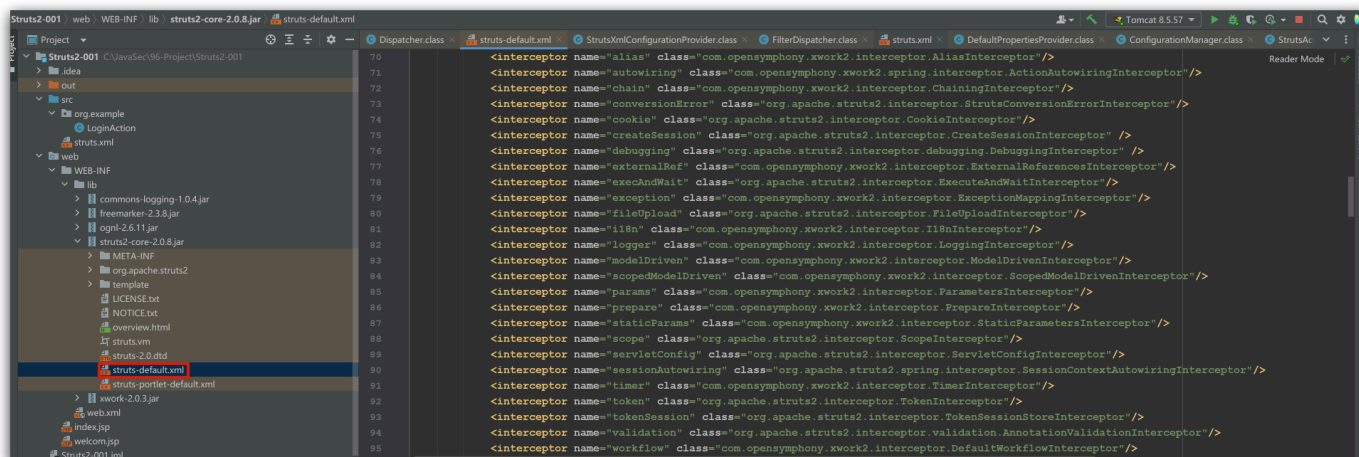
```

169 private void init_TraditionalXmlConfigurations() {
170     String configPaths = (String)this.initParams.get("config");
171     if (configPaths == null) {
172         configPaths = "struts-default.xml,struts-plugin.xml,struts.xml";
173     }
174
175     String[] files = configPaths.split( regex: "\\s*[,]\\s*" );
176     String[] arr$ = files;
177     int len$ = files.length;
178
179     for(int i$ = 0; i$ < len$; ++i$) {
180         String file = arr$[i$];
181         if (!file.endsWith(".xml")) {
182             throw new IllegalArgumentException("Invalid configuration file name");
183         }
184
185         if ("xwork.xml".equals(file)) {
186             this.configurationManager.addConfigurationProvider(new XmlConfigurationProvider(file, errorIfMissing: false));
187         } else {
188             this.configurationManager.addConfigurationProvider(new StrutsXmlConfigurationProvider(file, errorIfMissing: false, this.servletContext));
189         }
190     }
191 }

```

可以发现对配置文件进行逐个加载，三个文件的含义分别为：

1、struts-default.xml：在struts2-core-2.08.jar包中，是struts2默认的配置，里面定义了默认的bean、result、interceptor等内容



2、struts-plugin.xml：Struts2框架的插件的配置文件，在struts-xxx-plugin.jar中

3、struts.xml：用户自定义的struts配置文件，优先级大于struts-default.xml

总结下加载配置用到的类以及加载流程

- **ConfigurationProvider:配置元素加载器**

1. 真正解析配置文件的类

- **Configuration：配置元素**

1. 站在框架级别的视角管理所有配置元素
2. 对所有配置元素进行初始化调度

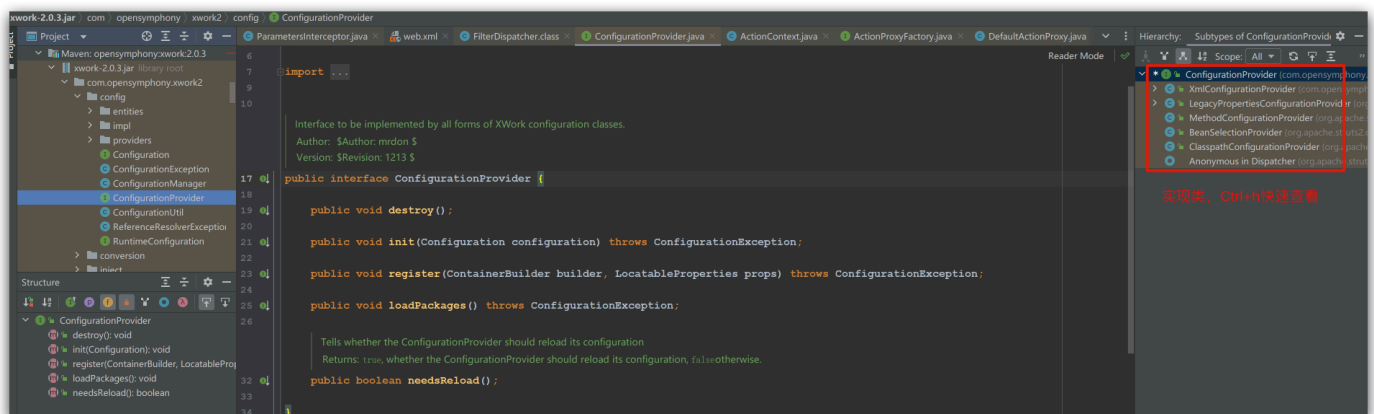
- **ConfigurationManager：配置元素管理器**

1. 内部封装了Configuration、ConfigurationProvider类，是整个配置管理的配置操作类

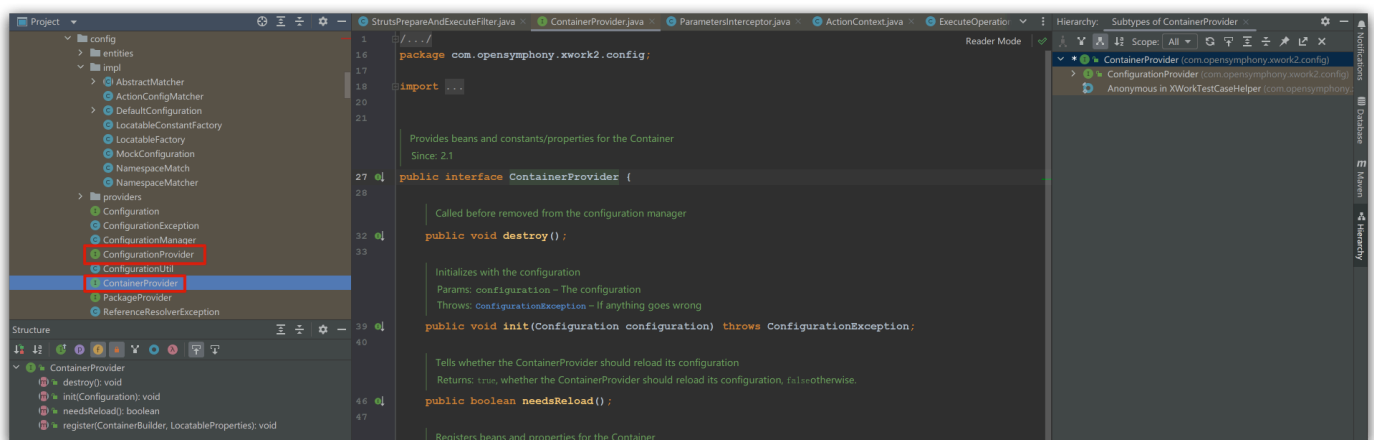
2.2、Struts2的配置管理

1. Provider-配置元素加载器（ContainerProvider容器加载器、PackageProvider事件映射加载器）
2. Configuration-配置管理（Configuration配置类、ConfigurationManager配置管理）

关于Provide在Struts2的2.0.x版本和2.2.x版本是有所区别的，例如2.0.x版本



2.2.x版本如下图，在该版本中ConfigurationProvider是空的，继承了ContainerProvider



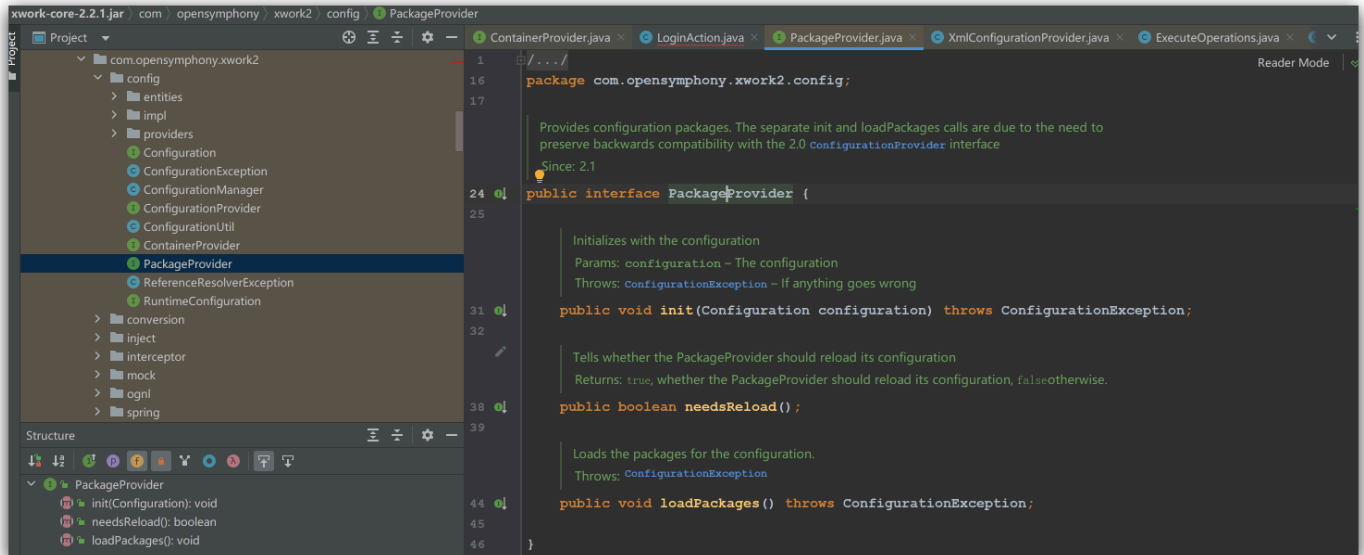
我们以2.0.x为版本说明，通过ConfigurationProvider为例，可以看到它的实现类为

- XmlConfigurationProvider：加载xml类配置文件
- LegacyPropertiesConfigurationProvider：加载properties类配置文件，如default.properties

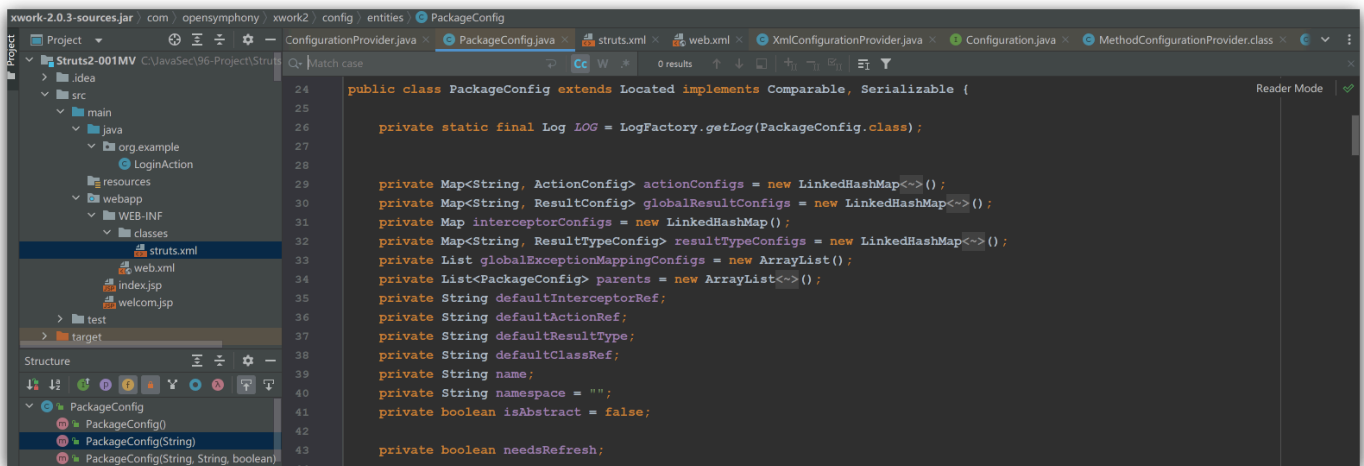
- MethodConfigurationProvider: 没怎么看明白方法的使用场景，但可以看出是找Action的方法名称
- BeanSelectionProvider: 将配置文件中的bean与Java类进行映射，初始化默认容器内置对象
- ClasspathConfigurationProvider: 默认配置，如JP后缀名

这些ConfigurationProvider在dispatch执行init方法时被调用，其中MethodConfigurationProvider未被调用

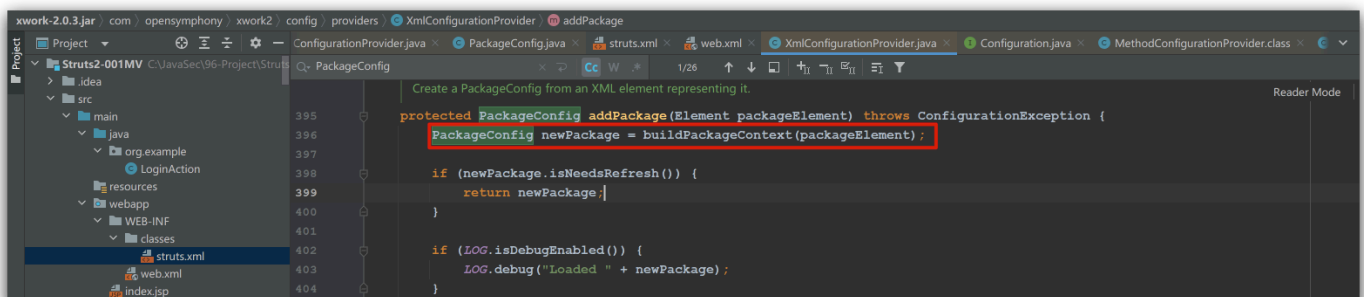
PackageProvider在2.0.x版本中并没有这个类，但在2.2.x版本也仅仅是一个接口，如下图，该接口已经被XmlConfigurationProvider等类实现

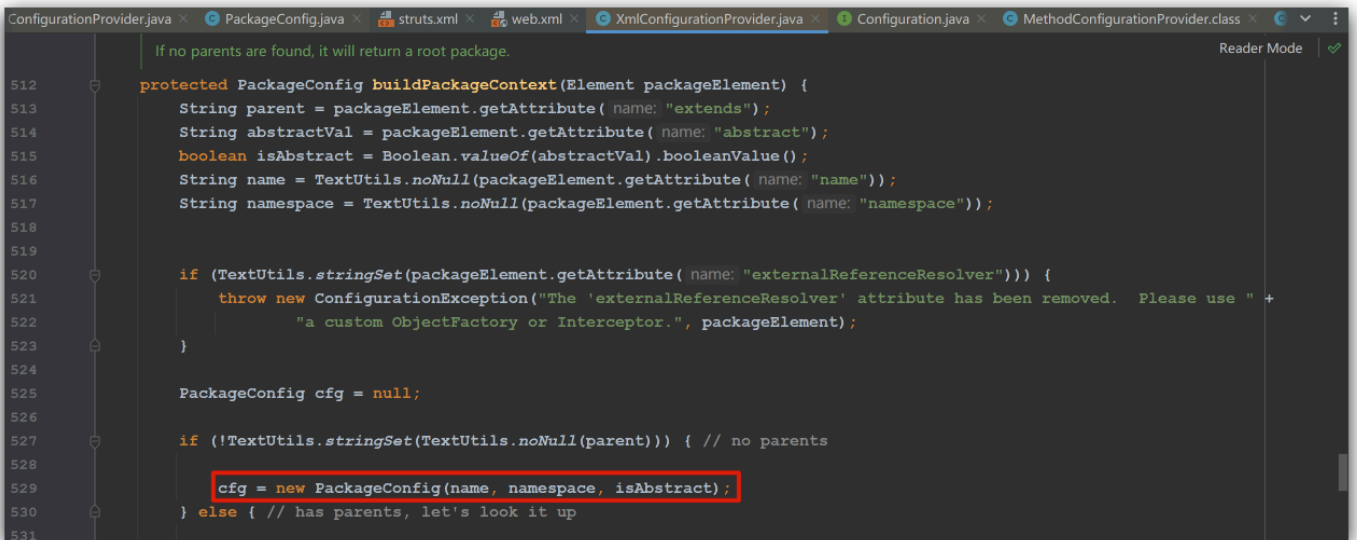


struts.xml在解析时对应的Java类是PackageConfig，类中的属性与xml一一对应



但在struts2.0.x中PackageConfig的创建是通过普通的新模式



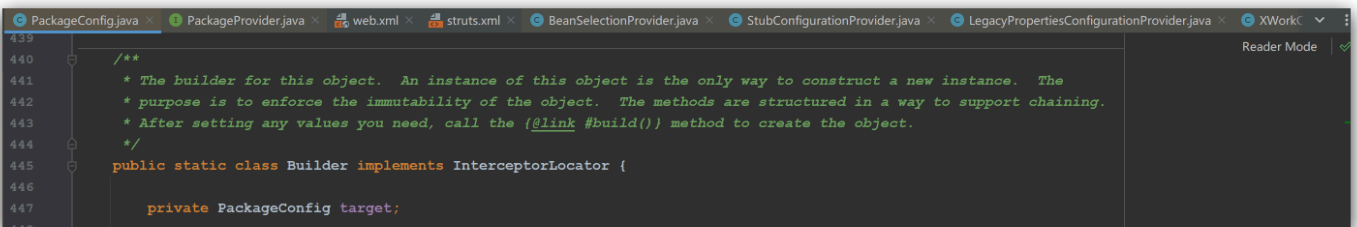


```

512     If no parents are found, it will return a root package.
513     protected PackageConfig buildPackageContext(Element packageElement) {
514         String parent = packageElement.getAttribute( name: "extends");
515         String abstractVal = packageElement.getAttribute( name: "abstract");
516         boolean isAbstract = Boolean.valueOf(abstractVal).booleanValue();
517         String name = TextUtils.noNull(packageElement.getAttribute( name: "name"));
518         String namespace = TextUtils.noNull(packageElement.getAttribute( name: "namespace"));
519
520         if (TextUtils.stringSet(packageElement.getAttribute( name: "externalReferenceResolver"))) {
521             throw new ConfigurationException("The 'externalReferenceResolver' attribute has been removed. Please use " +
522                 "a custom ObjectFactory or Interceptor.", packageElement);
523         }
524
525         PackageConfig cfg = null;
526
527         if (!TextUtils.stringSet(TextUtils.noNull(parent))) { // no parents
528             cfg = new PackageConfig(name, namespace, isAbstract);
529         } else { // has parents, let's look it up
530
531

```

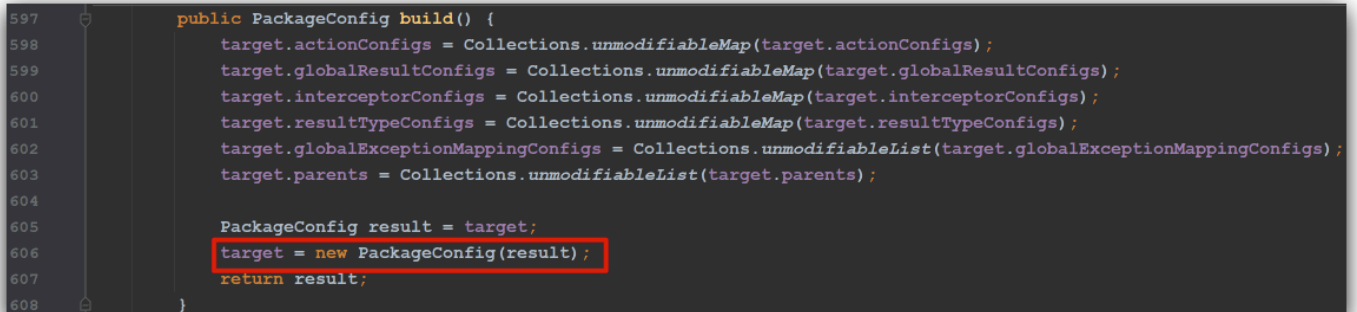
在struts2.2.x中PackageConfig的创建是通过build模式，PackageConfig的构造方法是protected（允许同包下的类和不同包的子类访问），体现了Package初始化的严格隔离。



```

439
440     /**
441      * The builder for this object. An instance of this object is the only way to construct a new instance. The
442      * purpose is to enforce the immutability of the object. The methods are structured in a way to support chaining.
443      * After setting any values you need, call the {@link #build()} method to create the object.
444      */
445     public static class Builder implements InterceptorLocator {
446
447         private PackageConfig target;
448
449

```



```

597     public PackageConfig build() {
598         target.actionConfigs = Collections.unmodifiableMap(target.actionConfigs);
599         target.globalResultConfigs = Collections.unmodifiableMap(target.globalResultConfigs);
600         target.interceptorConfigs = Collections.unmodifiableMap(target.interceptorConfigs);
601         target.resultTypeConfigs = Collections.unmodifiableMap(target.resultTypeConfigs);
602         target.globalExceptionMappingConfigs = Collections.unmodifiableList(target.globalExceptionMappingConfigs);
603         target.parents = Collections.unmodifiableList(target.parents);
604
605         PackageConfig result = target;
606         target = new PackageConfig(result);
607         return result;
608     }

```

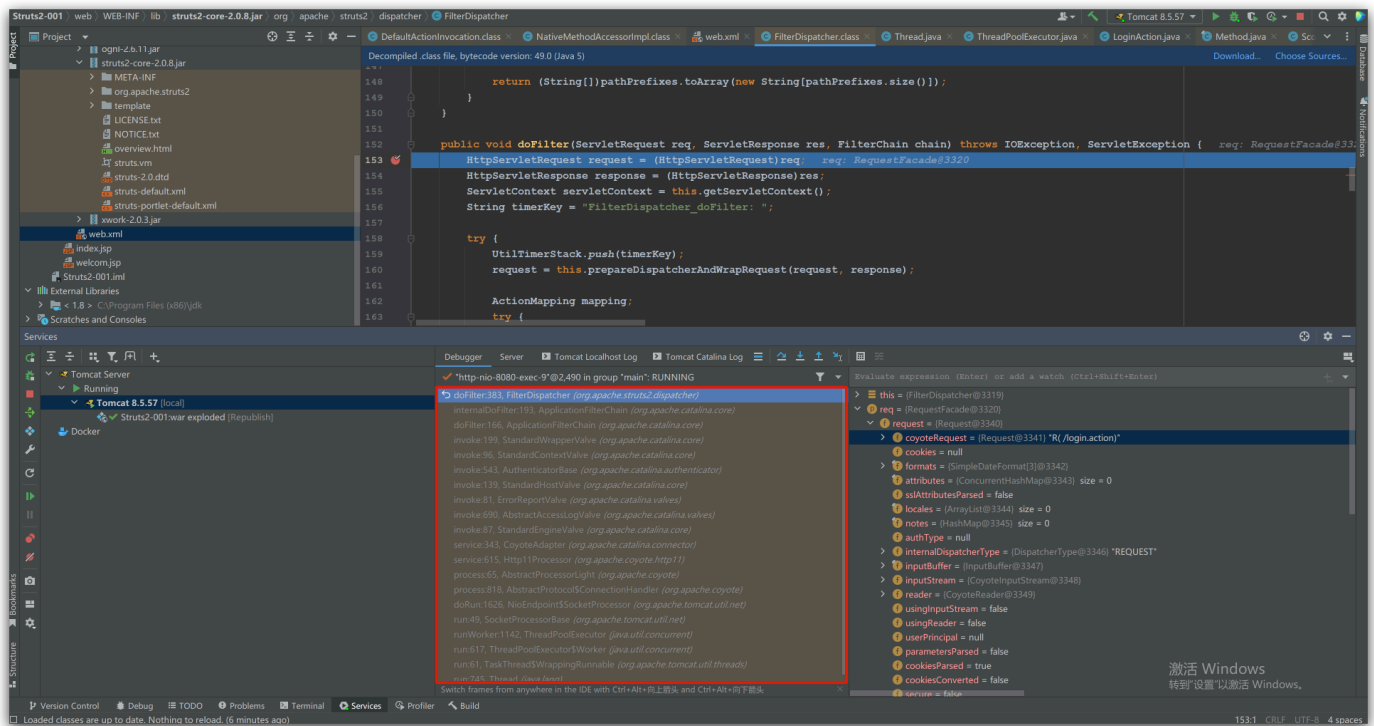
2.3、Struts2处理HTTP请求（FilterDispatcher）

FilterDispatcher是早期struts2的过滤器，后期的都用StrutsPrepareAndExecuteFilter了。

FilterDispatcher是struts2.0.x到2.1.2版本的核心过滤器。

StrutsPrepareAndExecuteFilter是自2.1.3开始就替代了FilterDispatcher的。

我们分析的Struts2-001的漏洞是2.0.8，所以在web.xml的Filter指向的便是org.apache.struts2.dispatcher.FilterDispatcher，我们可以在该类的doFilter方法第一行打断点，发起HTTP请求会断点在doFilter的第一行停下，如下图所示

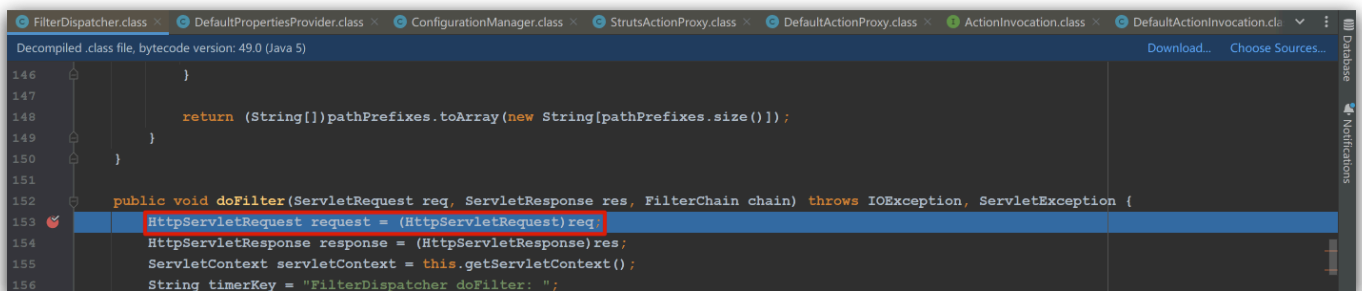


可以发现在doFilter之前代码全部属于org.apache.catalina.core，该包是Tomcat的核心包，所以可以得出以下结论：

org.apache.struts2.dispatcher.FilterDispatcher是Struts2的入口函数，所有的HTTP请求都经过这里。

我有个问题，Tomcat中接收到的HTTP对象及参数，如何传递到Struts2当中的？

通过分析doFilter方法可以看到，是将Tomcat传过来的ServletRequest转为HttpServletRequest，但参数中的ServletRequest其实是RequestFacade类，而该类是Tomcat封装的



Struts2.0.x如何处理HTTP请求？

1. 初始化：刚刚说明了Struts在启动时调用init方法对Struts的配置进行初始化，完成基础对象的构建，从而满足Struts后续的功能需求
2. 处理HTTP请求：在web.xml中配置的入口类是FilterDispatcher，该类的doFilter便是Web服务器将HTTP请求交给Struts的第一个方法，所以我们从doFilter方法开始分析，先看下核心代码

```
1 // 获取Servlet上线文对象，对象内主要有HTTP的一些核心元素，例如Session、Header、Method等
2 ServletContext servletContext = this.getServletContext();
3 // 准备request，封装request对象
4 request = this.prepareDispatcherAndWrapRequest(request, response);
5 // 下面是this.prepareDispatcherAndWrapRequest方法的主要逻辑
```

```

6      // 将dispatcher对象绑定到当前线程, this.dispatcher是ThreadLocal
7      Dispatcher.setInstance(this.dispatcher);
8      // 配置request编码等工作
9      this.dispatcher.prepare(request, response);
10     // 封装request对象为Struts的Request对象
11     request = this.dispatcher.wrapRequest(request, this.getServletContext());
12     // 获取ActionMapping, 这是一个非常简单的类, 用来表示请求的URL与Java对象的对应关系
13     mapping = actionMapper.getMapping(request,
14     this.dispatcher.getConfigurationManager());
14     // 开始执行URL对应的Action, 这里才是Struts处理HTTP请求的真正位置, 接下来由Dispatcher调用
    ActionProxy将请求转发
15     // 到xWork中处理
16     // 所以dispatch才是将HTTP请求与MVC的实现 (xWork框架) 分离的核心分发器
17     // 到达xWork之后, 就是ActionProxy、ActionInvocation、Interceptor的执行逻辑了
18     this.dispatcher.serviceAction(request, response, servletContext, mapping);
19     String resourcePath = RequestUtils.getServletPath(request);
20     //返回请求
21     chain.doFilter(request, response);

```

下图是wrapRequest方法主体, 如果content_type为multipart/form-data则封装为MultiPartRequest, 这代表有文件上传的请求, 如果是普通的HTTP请求, 则封装为StrutsRequestWrapper (HttpServletRequest的实现类), 那么为什么要进行封装? 直接用HttpServletRequest不行吗?

HttpServletRequest是什么? 它是Web服务器与Struts第一次沟通传递的参数, 里面包含了用户的HTTP请求。

首先我们需要看HttpServletRequest是从哪里来的, 它是通过doFilter方法传到Struts框架的, 所以它来自Web服务器, 而Struts要实现与Web服务器的解耦, 必须要对HttpServletRequest中的对象进行本地化操作, 才能实现解耦。

```

452     public HttpServletRequest wrapRequest(HttpServletRequest request, ServletContext servletContext) throws IOException
453     {
454         if (request instanceof StrutsRequestWrapper) {
455             return request;
456         } else {
457             String content_type = request.getContentType();
458             Object request;
459             if (content_type != null && content_type.indexOf("multipart/form-data") != -1) {
460                 MultiPartRequest multi = (MultiPartRequest) this.getContainer().getInstance(MultiPartRequest.class);
461                 request = new MultiPartRequestWrapper(multi, request, this.getSaveDir(servletContext));
462             } else {
463                 request = new StrutsRequestWrapper(request);
464             }
465             return (HttpServletRequest) request;
466         }
467     }

```

但是在Struts2整个执行过程中, 在doFilter函数开始的时候就已经知道了要请求的Action, 为什么还要做如此复杂的执行流程, 有以下几个问题:

1、interceptor可以对最终的Action做一些前置准备动作, 例如过滤参数, 但为什么要用ActionInvocation调用Interceptor?

2、为什么用ActionProxy做代理?

先说下ActionProxy与ActionInvocation作用:

1、ActionProxy对如何获取Action对象进行了封装

2、ActionInvocation对如何执行Action进行了封装

为了解决这个问题，我们先对比下ActionProxy和我们自己写的LoginAction

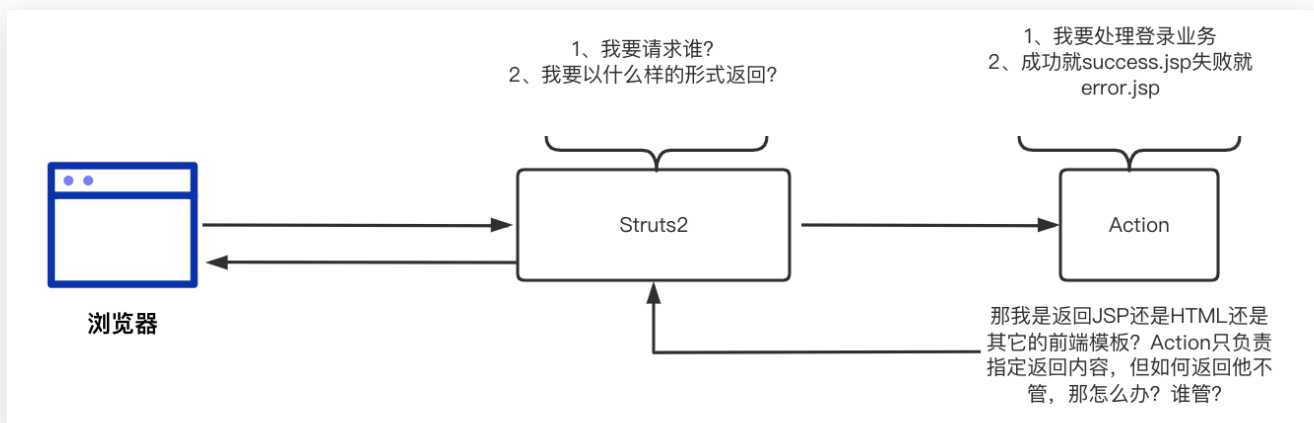
```
21 public class DefaultActionProxy implements ActionProxy, Serializable {
22     private static final long serialVersionUID = 3293074152487468527L;
23     private static final Log LOG = LogFactory.getLog(DefaultActionProxy.class);
24     protected Configuration configuration;
25     protected ActionConfig config;
26     protected ActionInvocation invocation;
27     protected UnknownHandler unknownHandler;
28     protected Map extraContext;
29     protected String actionName;
30     protected String namespace;
31     protected String method;
32     protected boolean executeResult;
33     protected boolean cleanupContext;
34     protected ObjectFactory objectFactory;
35     protected ActionEventListener actionEventListener;
```

```
public class LoginAction extends ActionSupport {

    public String execute() throws Exception {
        return "error";
    }

    no usages
    public String say(String msg) {
        System.out.println(msg);
        return "error";
    }
}
```

我们自己的LoginAction中只有业务逻辑，并没有任何关于业务之外的内容，而在ActionProxy中，却包含ActionInvocation、ActionName、method、executeResult等内容。我们在想一个问题：一个HTTP的请求逻辑是什么？



上面的图大致说明了HTTP请求Action的过程，Action只管业务，所以Action之前的过滤工作、Action之后的收尾工作，这些事情都得Struts2帮你做，所以就回答了刚开始的【为什么doFilter中已经拿到了要请求的Action，但为什么不直接调用】的问题，所以最终结果还是归结到【框架】两个字，这就是框架所能带给你的便利性，前可以过滤内容，后可处理返回，我们只需要关注中间的业务就行了。

在回顾下开始的两个问题：

问题一：interceptor可以对最终的Action做一些前置准备动作，例如过滤参数，但为什么要用ActionInvocation调用Interceptor？

答：可扩展性，如果代码写到ActionProxy中，对ActionProxy侵入性太大，不符合解耦思想；ActionProxy对如何获取Action对象进行了封装，ActionInvocation对如何执行Action进行了封装，两者各自负责自己的任务。清晰明了。

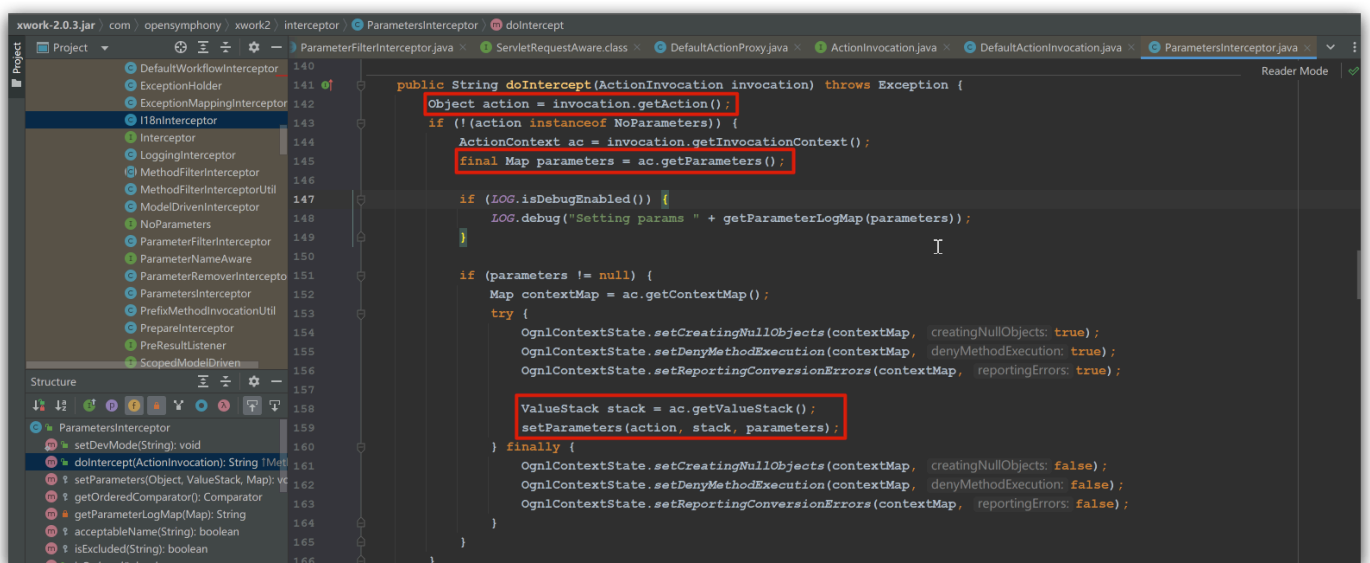
问题二：为什么用ActionProxy做代理？

答：使用了代理模式，增强业务Action的功能，也提高了程序的扩展性

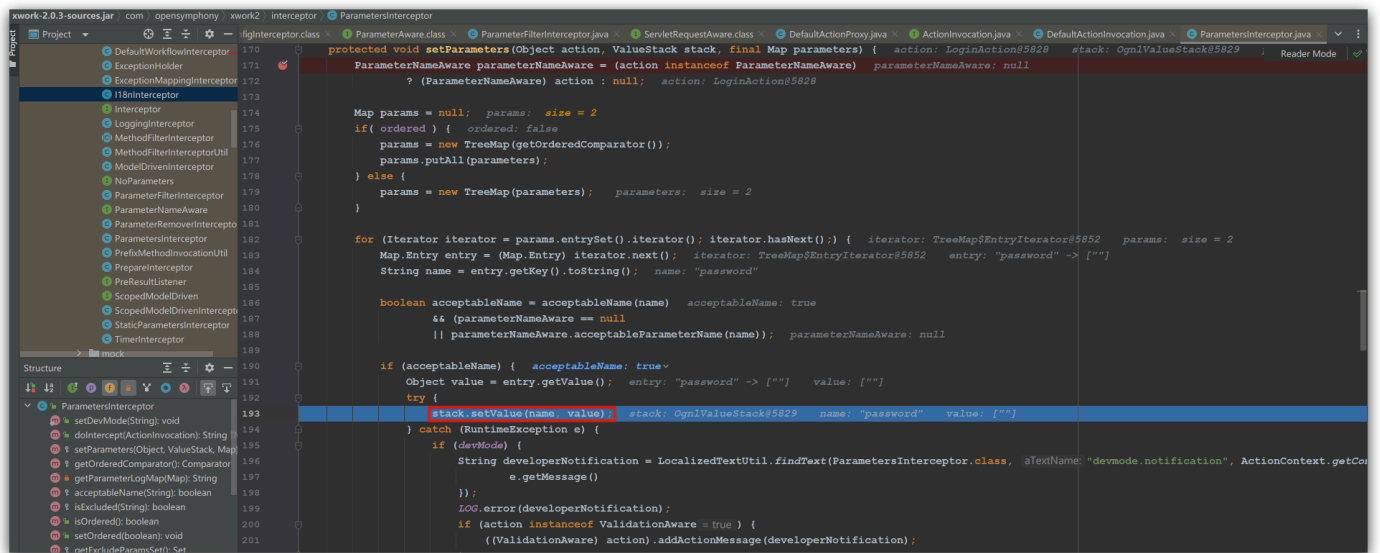
2.4、HTTP中的参数如何映射到Action中的？

当思考这个问题的时候，第一反应是通过反射，将Action类的属性反射修改，但Struts2并没有直接这么做，当然最终的原理还是通过反射修改。我们来看下Struts2怎么做的。

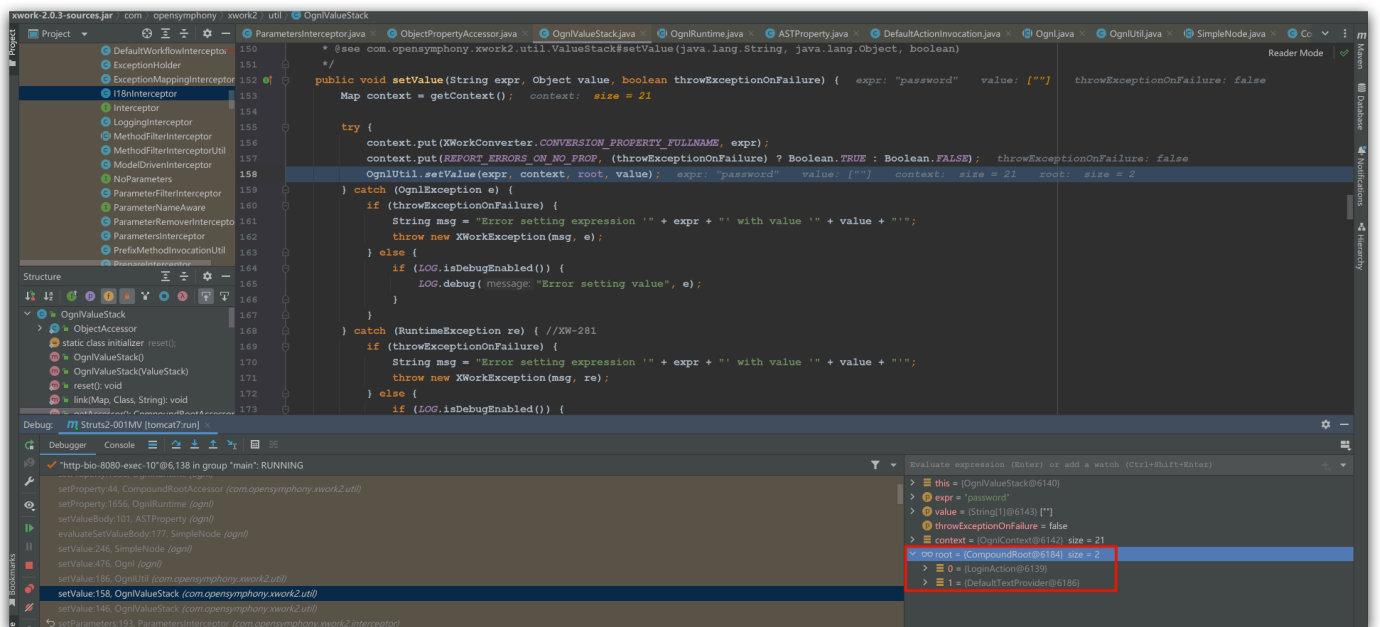
Struts2是通过ParametersInterceptor类，该类首先从ActionContext中取出ValueStack



然后使用valuestack的setValue方法，将HTTP参数映射到Action的属性，这就变成了ValueStack的操作

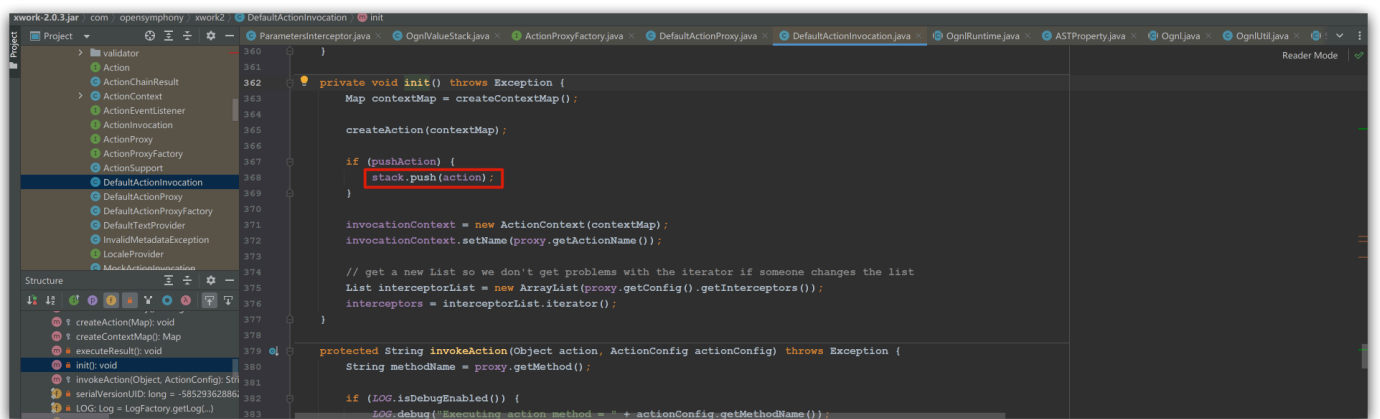


ValueStack中是有Action对象的，如下图，所以最终是ValueStack操作的Action对象



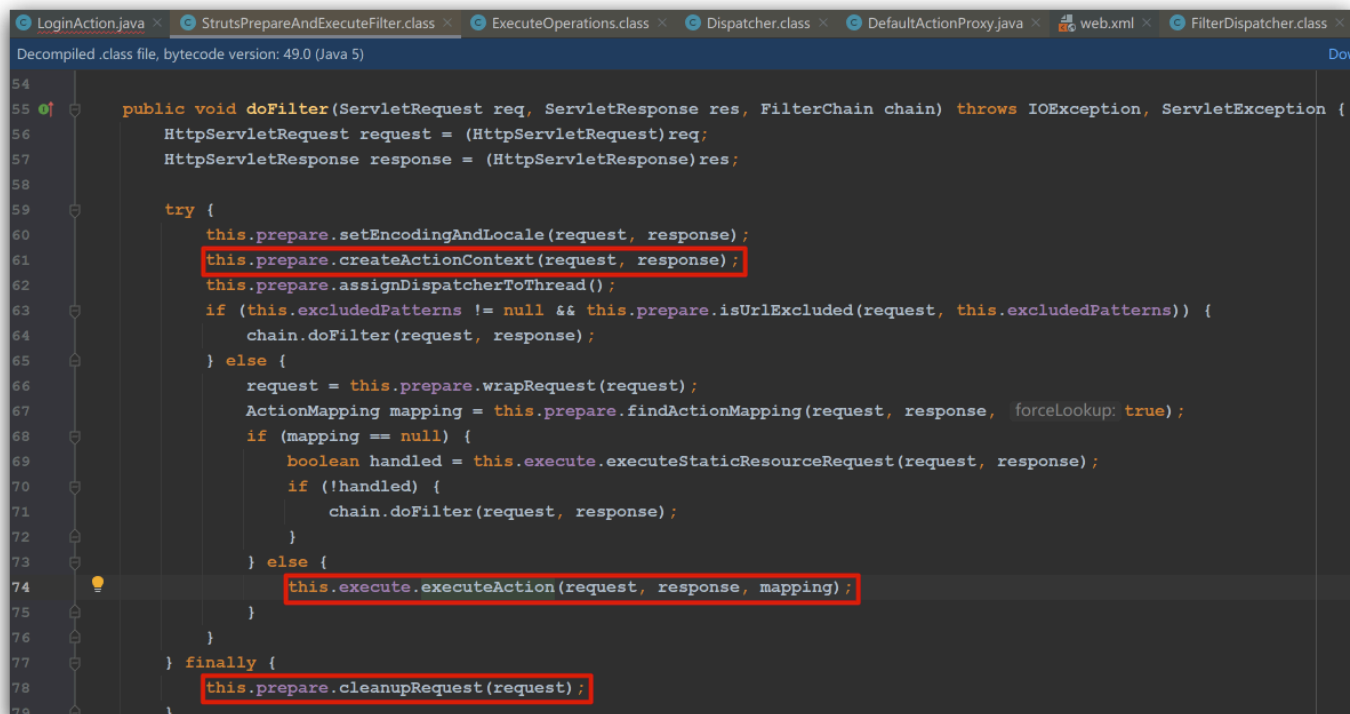
为什么Action对象在ValueStack中？

因为在DefaultActionInvocation类的init方法中，将Action放入了ValueStack

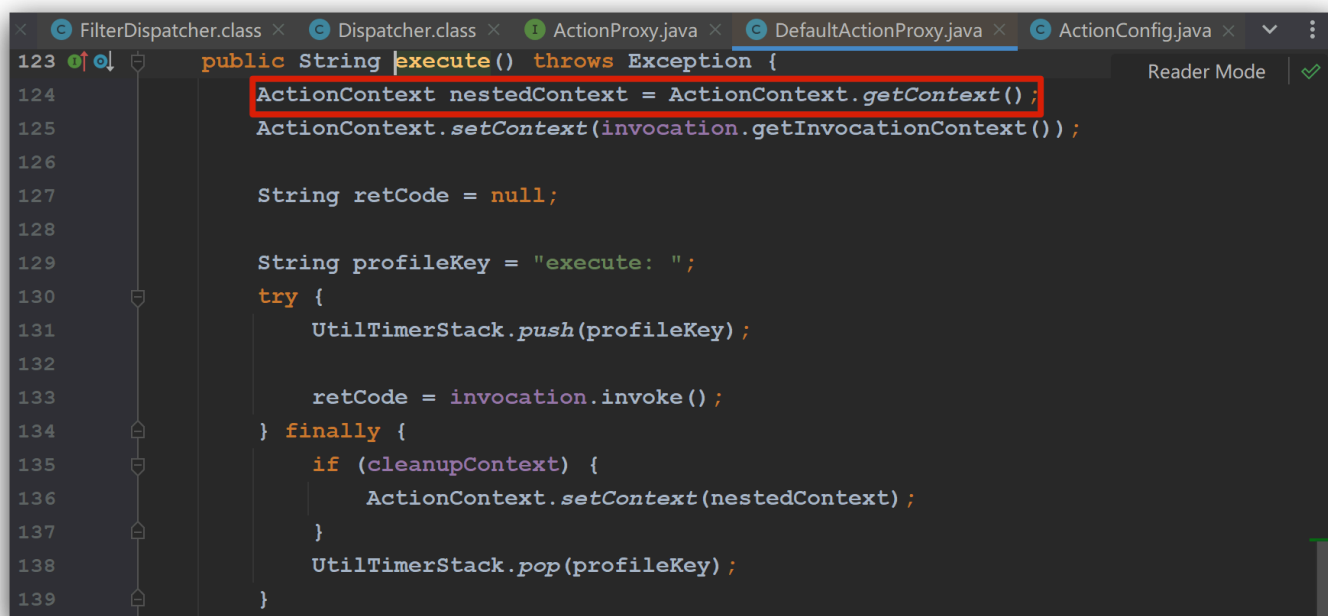


2.5、ActionContext与Action?

(Struts2.2.1版本中) ActionContext的创建在Action执行之前, Action执行之后便cleanup, 由此可以看出ActionContext是贯穿整个ActionContext生命周期的, 对当前执行的线程都有效。



在最初的2.0.x版本也是这个思想, 但代码有所区别, 如下图的DefaultActionProxy中的execute方法, 直接获取了ActionContext, 说明在此之前已经完成初始化。



ActionInvocation是将Action和Interceptor捆绑在一起调度, Action和Interceptor共同组成为XWork的执行栈, 是核心调度器, DefaultActionInvocation类是默认的实现类, 看下它的invoke方法

```

210 public String invoke() throws Exception {
211     String profileKey = "invoke: ";
212     try {
213         UtilTimerStack.push(profileKey);
214
215         if (executed) {
216             throw new IllegalStateException("Action has already executed");
217         }
218
219         if (interceptors.hasNext()) {
220             final InterceptorMapping interceptor = (InterceptorMapping) interceptors.next();
221             UtilTimerStack.profile( name: "interceptor: "+interceptor.getName(),
222                 () - {
223                     resultCode = interceptor.getInterceptor().intercept( invocation: DefaultActionInvocation.this );
224                     return null;
225                 });
226         } else {
227             resultCode = invokeActionOnly();
228         }
229     }
230 }

```

递归调用

此处调用真正的业务Action

方法内通过调用interceptor的intercept方法来调用各种拦截器，为处理最终的业务Action做准备，是一种递归调用，在intercept方法中将自己（this）传进去，然后调用自己的invoke方法实现递归

```

172 public String intercept( ActionInvocation invocation ) throws Exception {
173     String result;
174
175     try {
176         result = invocation.invoke();
177     } catch (Exception e) {
178         if (logEnabled) {
179             handleLogging(e);
180         }
181         List exceptionMappings = invocation.getProxy().getConfig().getExceptionMappings();
182         String mappedResult = this.findResultFromExceptions(exceptionMappings, e);
183         if (mappedResult != null) {
184             result = mappedResult;
185             publishException(invocation, new ExceptionHolder(e));
186         } else {
187             throw e;
188         }
189     }
190
191     return result;
192 }

```

而DefaultActionInvocation执行完invokeActionOnly(也就是调用业务Action)之后，可能还需要做其他的工作，该工作由resultListeners完成

```

228 } else {
229     resultCode = invokeActionOnly();
230 }
231
232 // this is needed because the result will be executed, then control will return to the Interceptor, which will
233 // return above and flow through again
234 if (!executed) {
235     if (preResultListeners != null) {
236         for (Iterator iterator = preResultListeners.iterator();
237             iterator.hasNext();) {
238             PreResultListener listener = (PreResultListener) iterator.next();
239
240             String _profileKey="preResultListener: ";
241             try {
242                 UtilTimerStack.push( _profileKey );
243                 listener.beforeResult( invocation: this, resultCode );
244             } finally {
245                 UtilTimerStack.pop( _profileKey );
246             }
247         }
248     }
249
250     // now execute the result, if we're supposed to
251     if (proxy.getExecuteResult()) {
252         executeResult();
253     }
254 }

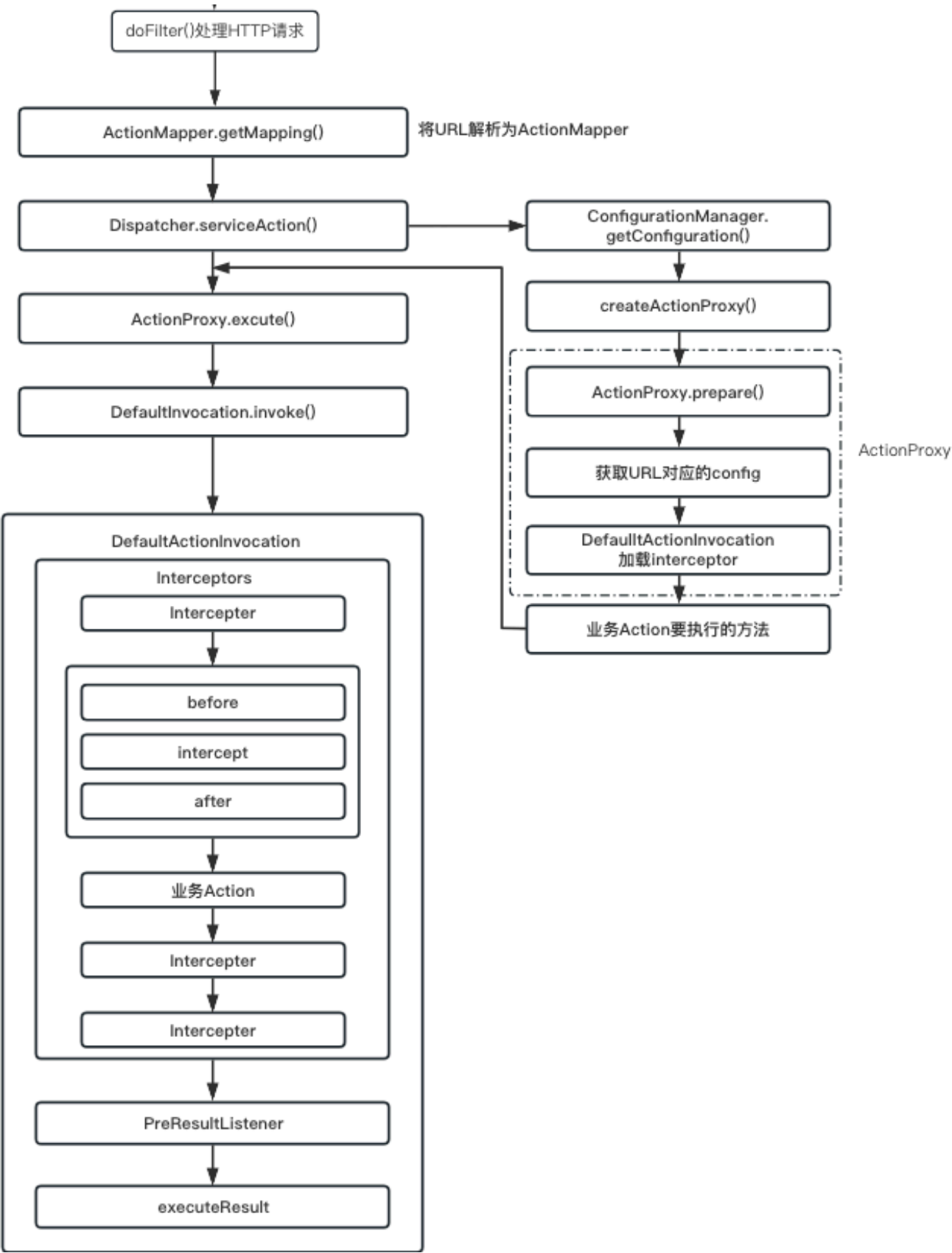
```

执行业务Action

业务Action之后的动作

返回结果

在整个DefaultActionInvocation中，Interceptor、Action、PreResultListener、Result四大元素共同构成了ActionInvocation执行的对象。而一个Interceptor由before、interceptor、after组成，在调用intercept方法时，自动调用before、invocation.invoke、after，如MessageStoreInterceptor类的interceptor方法，如下图



2.6、Action处理结束后的动作

通过DefaultActionInvocation的执行流程图，可以看到最后是有PreResultListener，这是Struts2预留的Action处理完之后，进行扩展的地方，不在展开讲了，先留个坑。

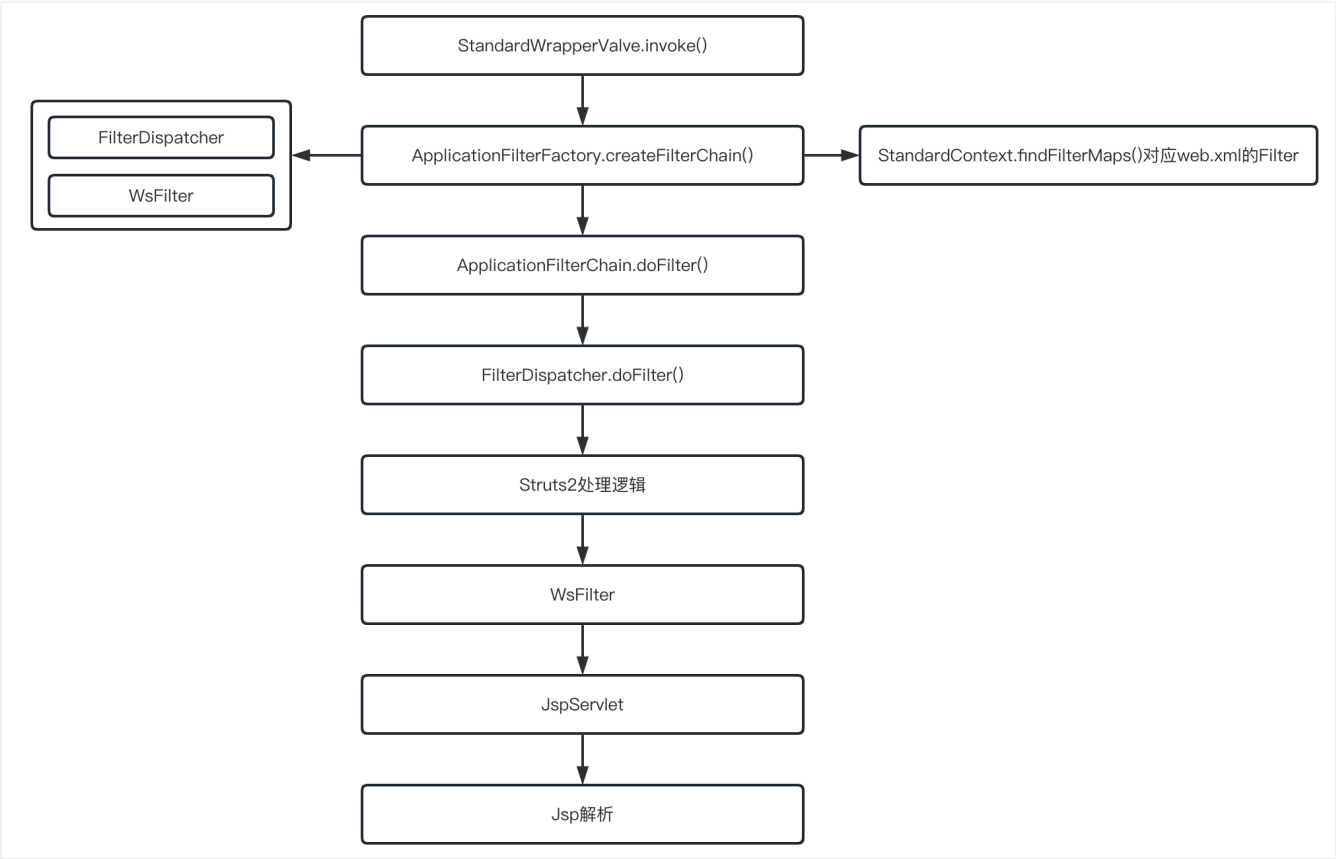
2.7、Response页面跳转

DefaultActionInvocation使用executeResult方法处理返回，经过代码调试，发现Struts2使用dispatcher.forward实现跳转。那么谁实现的页面的跳转，如何做的跳转？是真的URL跳转吗？还是转到其他类解析JSP了？跳转后如何开始解析的JSP？

要回答这个问题首先看下Tomcat简易的运行原理，ApplicationFilterFactory创建了2个Filter（该类在tomcat-catalina-xx.jar的org.apache.catalina.core包）

- org.apache.struts2.dispatcher.FilterDispatcher (web.xml中定义)
- org.apache.tomcat.websocket.server.WsFilter

ApplicationFilterChain负责循环调用这两个Filter，该类在tomcat-catalina-xx.jar的org.apache.catalina.core包



1. FilterDispatcher：用来调度执行Action，如果忘了可以看上面的FilterDispatcher运行的结构，这个FilterDispatcher最后会dispatcher.forward()到某个JSP中
2. WsFilter：这个类是Tomcat的类(maven配置如下)，是属于WebSocket的类，FilterDispatcher执行完后会接着执行WsFilter判断是不是WebSocket请求，如果不是则跳转到JSP所对应的JSPServlet中处理，这时才真正的开始解析JSP

```

1 <!-- https://mvnrepository.com/artifact/org.apache.tomcat/tomcat-websocket -->
2 <dependency>
3   <groupId>org.apache.tomcat</groupId>
4   <artifactId>tomcat-websocket</artifactId>
5   <version>8.0.3</version>
6   <scope>provided</scope>
7 </dependency>

```

但struts2是跳转到JSPServlet还是其它地方，就需要看Struts2集成了哪个View框架

三、Struts2的核心对象

3.1、Container容器

XWork的容器Container，在xwork.jar的com.opensymphony.xwork2.inject.Container类

为什么要用容器管理：引入一个与具体业务逻辑完全无关的容器来进行对象的生命周期管理

容器的特点：

1. 应该是全局的、统一的对象
2. 最大程度降低容器对业务逻辑的入侵
3. 提供简单而且全面的对象操作接口

容器的常见操作方法：

1. 获取对象实例：getInstance、getInstanceName
2. 处理对象依赖关系：inject
3. 处理对象的作用域：setScopeStrategy、removeScopeStrategy

容器在系统初始化时进行初始化，在任何代码位置都可以对全局的容器进行访问

看下struts.xml配置文件内容

```

1 <?xml version="1.0" encoding="UTF-8" ?>
2 <!DOCTYPE struts PUBLIC
3   "-//Apache Software Foundation//DTD Struts Configuration 2.3/EN"
4   "http://struts.apache.org/dtds/struts-2.3.dtd">
5
6 <struts>
7   <!--bean节点定义框架级别的内置或自定义对象 -->
8   <bean class="com.opensymphony.xwork2.ObjectFactory" name="xwork" />
9   <!--<constant>元素用常量的配置-->
10  <constant name="struts.enable.DynamicMethodInvocation" value="false" />
11  <constant name="struts.devMode" value="true" />
12  <!--<package>元素用于包配置-->
13  <package name="default" namespace="/" extends="struts-default">
14    <!--配置Action-->
15    <action name="index" class="Xxx" />
16    <!--配置Result-->
17    <result type="dispatcher">

```

```
18         <param name="location">/index.jsp</param>
19     </result>
20 </action>
21 </package>
22 <!-- <include>元素用于包含配置 -->
23 <include file="example.xml"/>
24 </struts>
```

Struts.properties配置文件内容

```
1 # 编码
2 struts.i18n.encoding=UTF-8
3 # action的后缀
4 struts.action.extension=action
```

Xwork通过调用getInstance方法获取对象实例，这些实例来自哪里？通过struts的配置文件可以看出有这么几类：

1. bean节点：框架内部对象、开发者自定义对象
2. package节点：Action事件对象
3. constant与properties文件：系统运行参数

通过容器获取上面的实例，不仅对象自身实例化，且对象所依赖的对象也会被依赖注入而进行实例化，这就是容器管理的好处，而这个操作就是Container.inject()方法实现的，它能帮助我们管理这些实例化的对象，从而使任意对象与容器、框架关联起来。

xwork会扫描对象内部中具有@Inject这个Annotation的属性、方法、构造方法、方法中的参数，并将它们注入Container容器。

Inject是个接口，在xwork.jar的com.opensymphony.xwork2.inject

```

17 package com.opensymphony.xwork2.inject;
18
19 import ...
20
21
22 |
23 | Annotates members and parameters which should have their value[s] injected.
24 | Author: crazybob@google.com (Bob Lee)
25 |
26
27 @Target({METHOD, CONSTRUCTOR, FIELD, PARAMETER})
28 @Retention(RUNTIME)
29 public @interface Inject {
30
31     |
32     | Dependency name. Defaults to Container.DEFAULT_NAME.
33     |
34     String value() default DEFAULT_NAME;
35
36     |
37     | Whether or not injection is required. Applicable only to methods and fields (not constructors or
38     | parameters).
39     |
40     boolean required() default true;
41 }

```

例如下图的ActionSupport类，getTextProvider中使用container容器的前提是容器存在，因为@Inject的注解，所以使得ActionSupport初始化时被注入container对象，而getTextProvider是使用期间被调用，此时全局的容器对象就可以被随意调用完成依赖注入，也就是container.inject(tpf)。

```

private TextProvider getTextProvider() {
    if (this.textProvider == null) {
        TextProviderFactory tpf = new TextProviderFactory();
        if (this.container != null) {
            this.container.inject(tpf);
        }

        this.textProvider = tpf.createInstance(this.getClass(), provider: this);
    }

    return this.textProvider;
}

@Inject
public void setContainer(Container container) { this.container = container; }

```

那么开发者在自定义的代码中如何获取容器中对象？可以借助@Inject来实现

```

import com.opensymphony.xwork2.ObjectFactory;
import com.opensymphony.xwork2.inject.Inject;

no usages
public class Test {
    1 usage
    private ObjectFactory objectFactory;

    no usages
    @Inject
    public void setObjectFactory(ObjectFactory objectFactory) {
        this.objectFactory = objectFactory;
    }
}

```

XWork的Container容器是如何实现对象管理的呢？

首先来看下Container的实现类，

```

1  class ContainerImpl implements Container {
2      final Map<Key<?>, InternalFactory<?>> factories;
3      final Map<Class<?>, Set<String>> factoryNamesByType;
4      ContainerImpl(Map<Key<?>, InternalFactory<?>> factories) {
5          this.factories = factories;
6          // 定义一个HashMap, 存放每个type (对应的是struts.xml中bean元素的type) 下的所有类名
7          Map<Class<?>, Set<String>> map = new HashMap<Class<?>, Set<String>>();
8          for (Key<?> key : factories.keySet()) {
9              Set<String> names = map.get(key.getType());
10             if (names == null) {
11                 names = new HashSet<String>();
12                 map.put(key.getType(), names);
13             }
14             names.add(key.getName());
15         }
16
17         for (Entry<Class<?>, Set<String>> entry : map.entrySet()) {
18             entry.setValue(Collections.unmodifiableSet(entry.getValue()));
19         }
20
21         this.factoryNamesByType = Collections.unmodifiableMap(map);
22     }
23     .....
24
25 }

```

下图是factories的KEY，是一个name和type，正好对应struts.xml的bean元素的name和type

```

1 class Key<T> {
2     // 对应的是struts.xml中bean元素的type
3     final Class<T> type;
4     // 对应的是struts.xml中bean元素的name
5     final String name;
6     final int hashCode;
7 }

```

下图是factories的Value，只有create方法，也就是说一旦实现这个接口，就要调用create方法创建指定的对象

```

1 interface InternalFactory<T> extends Serializable {
2     T create(InternalContext context);
3 }

```

由此可见factories中保存的是Java对象的构建方法，而不是真正的对象，所以容器的本质是一个工厂。

factories保存了所有需要实例化的Java类，但前面讲到的依赖注入是：不仅对象自身实例化，且对象所依赖的对象也会被依赖注入而进行实例化。那么两者的关系是如何维护的？这个关系应该是一对一或者一对多的关系。

ContainerImpl类中的injectors变量如下，该变量的key为class，值为满足条件的Injector，以此Map维护两者之间的关系。

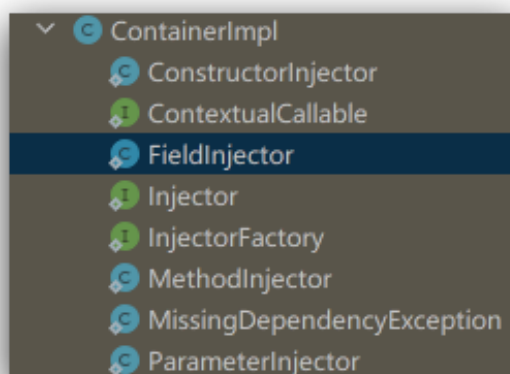
ReferenceCache是一个缓存类，当操作Map的get方法时会判断是否有缓存，没有则去注入。

```

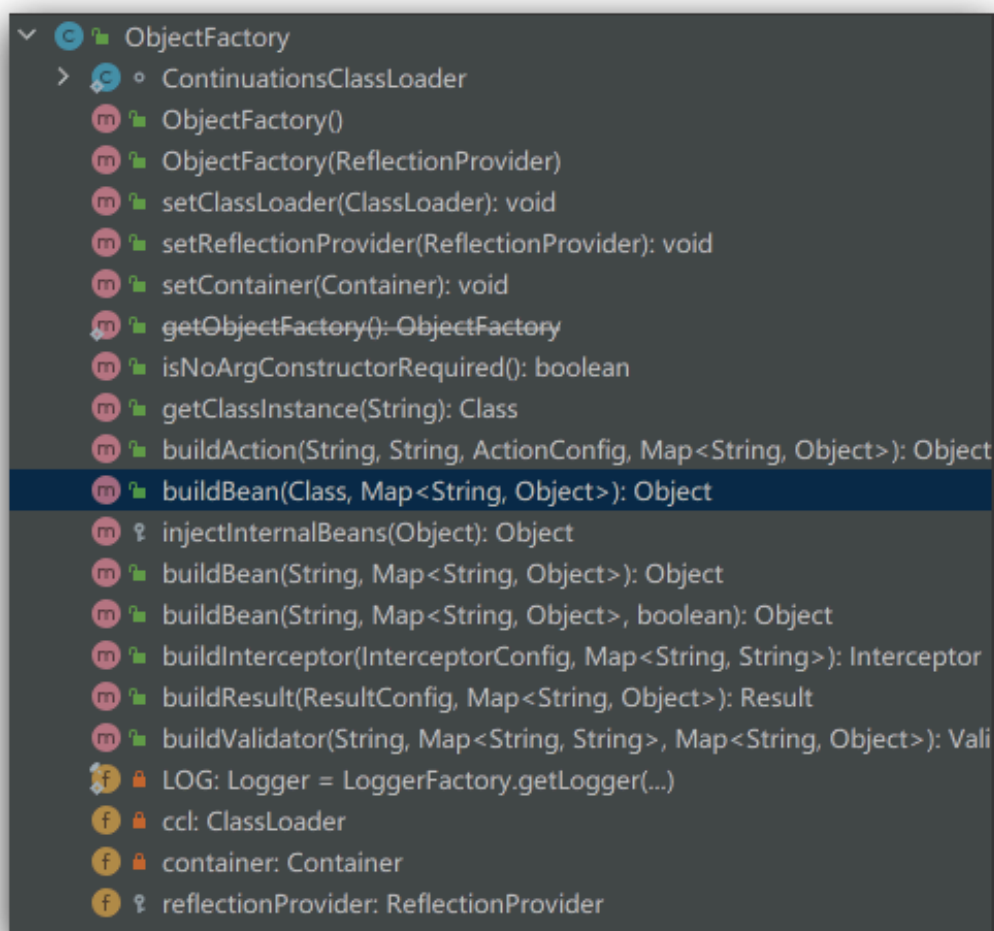
1 final Map<Class<?>, List<Injector>> injectors =
2     new ReferenceCache<Class<?>, List<Injector>>() {
3         @Override
4         protected List<Injector> create(Class<?> key) {
5             List<Injector> injectors = new ArrayList<Injector>();
6             addInjectors(key, injectors);
7             return injectors;
8         }
9     }
10 };

```

当使用@Inject时，可以将其放在属性、方法、参数中，对于开发者而言感觉不到什么异常，只是觉得哪里需要注入便在哪里写@Inject，但在ContainerImpl类中则区分了不同的注入方法，如下图



上面的内容我们说过，Container本质是一个工厂，并没有负责对象的创建，只是维护了type下有哪些类、类与依赖类的关系，而真正类的创建是由ObjectFactory（xwork.jar的com.opensymphony.xwork2下）完成的，下图是它所有的方法



这些方法可以归为两类：

1. 构建XWork框架内部对象的方法：Action、Interceptor、Result、Validator
2. 构建普通bean对象的方法：buildBean

buildBean方法是XWork框架创建对象的唯一方法，这保证了所有XWork框架中的执行对象都受到XWork容器的管理

那Container与ObjectFactory有什么区别？

1. Container是Struts2初始化时候创建的，从而在运行期间可以获取一个定义好的内置对象，以取为主
2. ObjectFactory则是运行过程中构建对象，提供了一个与Container交互的窗口，是以建为主

3.2、ActionContext

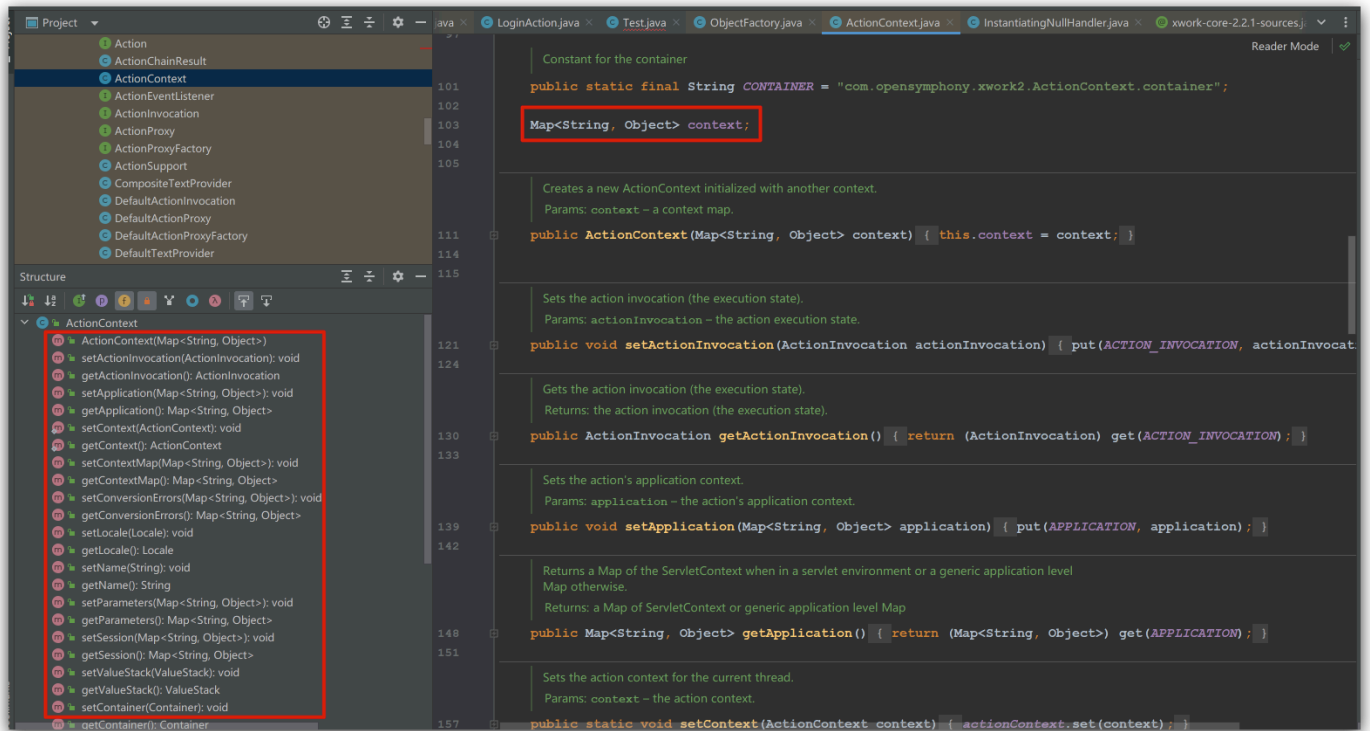
ActionContext是Action的上下文，Struts2自动在其中保存了一些在Action执行过程中所需的对象，比如session, parameters, locale等。Struts2会根据每个执行HTTP请求的线程来创建对应的ActionContext，即一个线程有一个唯一的ActionContext。因此，使用者可以使用静态方法ActionContext.getContext()来获取当前线程的ActionContext，也正是由于这个原因Action是线程安全的。

通过阅读ActionContext的源码可以发现，它存储的内容全部在Map类型的context变量中，而且提供了读取的方法。例如：

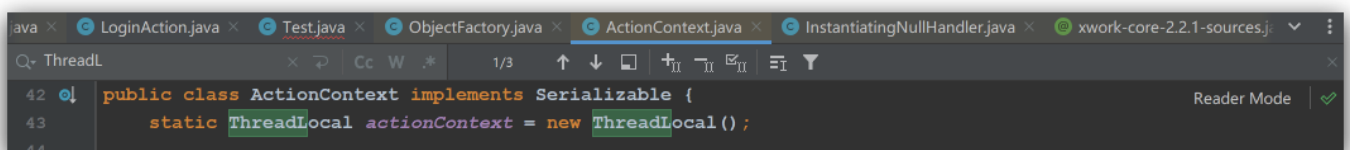
对XWork框架对象的访问：getContainer、getValueStack、getActionInvocation

对数据对象的访问：getSession、getApplication、getParameters等

这些访问接口包含了几乎所有整个Action请求中能用到的对象，但仔细观察可以发现它的数据访问接口参数都是Map类型，而不是HTTPSession、ServletContext这些纯正的Web容器对象，Map类型可以放所有的对象类型，看似不是很合理，但这就是ActionContext设计的精妙之处，因为XWork要与Web容器解耦，如果数据访问接口的参数是任意的Web类型，那么便与解耦思想违背了。



既然是用作数据共享，那么势必要面临线程安全问题，如果这个数据是类的内部实例变量，在多线程场景下就有可能存在线程安全问题，通过代码就可以看出来，内部封装了一个ThreadLocal对象，该对象放了当前线程的ActionContext对象



```
ava x LoginAction.java x Test.java x ObjectFactory.java x ActionContext.java x InstantiatingNullHandler.java x xwork-core-2.2.1-sources.j
Q- actionContext x Cc W .* 3/3 ↑ ↓ ↺ ↻ ↵ ↶ ↷
152 /**
153  * Sets the action context for the current thread.
154  *
155  * @param context the action context.
156  */
157 public static void setContext(ActionContext context) {
158     actionContext.set(context);
159 }
160
161 /**
162  * Returns the ActionContext specific to the current thread.
163  *
164  * @return the ActionContext for the current thread, is never <tt>null</tt>.
165  */
166 public static ActionContext getContext() {
167     return (ActionContext) actionContext.get();
168 }
169 // Don't do lazy context creation, as it requires container; the creation of which may
170 // precede the context creation
171 //if (context == null) {
172 //    ValueStack vs = ValueStackFactory.getFactory().createValueStack();
173 //    context = new ActionContext(vs.getContext());
174 //    setContext(context);
175 //}
176
177 }
```

ActionContext既然是数据载体，那么一定要被外部访问，其中就包括Web容器，这是通过ServletActionContext实现的，而且这个类时是Struts.jar中，而不是在xwork.jar中，这也代表了解耦思想，因为struts是与web容器打交道，xwork是核心，所以将负责与web容器打交道的ServletActionContext放在了struts.jar中，下图是Web原生对象与XWork封装后的对象的对应关系，ActionContext中保存的是并非web元素对象，而是经过封装的Struts2对象，dispatcher.createContextMap方法中有体现

Web原生对象	Struts2封装对象
HttpServletRequest	RequestMap
HttpSession	SessionMap
ServletContext	ApplicationMap

3.3、其它的核心对象

还有valueStack、Action、ActionProxy、ActionInvocation、Interceptor这些核心的类，在前面的内容已经顺带讲过，不再单独讲。

四、Struts2整体流程图

